COMP0005 Algorithms: Group Coursework Report

Group Number: XXX

Academic year 2025/26

1 Experimental framework overview

(Section 1 should be about one page in length.)

1.1 Framework design and architecture

The framework empirically measures the asymptotic relationship between the operation (search, insertion) times and number of nodes in a balanced tree.

Synthetic keys are generated in the $TestDataGenerator$, which contains four generators to suitably stress test the trees: sequential (ascending, descending), and random (uniform, normal). Uniform is the best case, as keys are naturally self balancing, (expected height of a vanilla BST is $O(logn)$ **algorithms**). Normal is the average case, as due to Central Limit Theorem, natural data often takes a normal distribution. Sequential are the worst cases, where maximal rebalances occur due to always inserting on the left or right of the tree.

The time to compare keys is a control variable, so the framework ensures all keys are a constant string length, k. This makes the time to compare keys $O(k)$. As k is constant, the cost of key comparisons is $O(1)$ with respect to the number of nodes, n, in the tree. This ensures the theoretical time complexity of operation is still $O(logn)$. As keys are generated numerically and cast to strings, the intended random distributions change due to the difference in numerical and lexicographic order. To maintain numerical ordering, truncating (casting to Python int), casting to str, and left padding with 0's is used. The string length of 7 was chosen as it allows for 10^7 unique keys, more than the number of nodes being inserted.

Data is collected using the $ExperimentalFramework$. The algorithm builds m trees up to the maximum size n while measuring operation cost at intervals defined on a geometric progression, ratio $r = e^{ln(n)/data\ points}$. The geometric progression spreads the data points equally when plotting a log-graph. To minimize bias from OS processes and cache effects, m is sufficiently large. To measure the asymptotic relationship, n is sufficiently large. To be confident in our data, r is sufficiently low. At each interval, an insertion is measured along with a search for a key inside and outside the tree.

After collecting the data, it is processed using the $StatisticsFramework$. Log values of n at each interval are calculated as this transforms the log relationship to a linear one. The framework then calculates the PMCC, a quantitative measurement of linearity, to verify the relationship. The regression line is calculated as it can be used to reason about the hidden constants of different tree implementations. The RMSE is calculated as it allows for

reasoning about the predictability of a tree implementation's performance. The residuals are plotted as they can reveal hidden mechanics about the trees (explored in section 2.1).

Finally, graphs are processed using Matplotlib.

1.2 Hardware/software setup for experiments

Hardware:

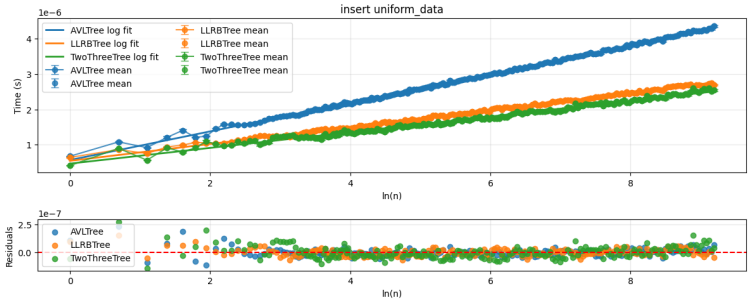
- Laptop: Apple MacBook Pro M4 (10 core CPU, 10 core GPU)
- Memory: 16GB LPDDR5X; 120 Gbs^{-1} bandwidth **wikiM4**
- Cache (`$ sysctl -a | grep perflevel`):
  - L1: 192 (instruction) + 128 KiB (data) per P core; 128 + 64 KiB per E core
  - L2: 16 MiB for P core cluster; 4 MiB for E core cluster

Software:

- OS: MacOS Tahoe 26.3.1
- Python Interpreter: 3.11.0
- Python Libraries: timeit, random, matplotlib

2 Performance results

(Section 2 should be about two to two-and-a-half pages in length.)



AVLTree: The regression statistics are:
The model of best fit is: y = 5.690849501503539e-07 + 4.040408525958418e-07*log(x)
The PMCC is: 0.9990170965506187
The coefficient of determination (r^2) is: 0.9980351592004281
The RMSE is: 3.709894630319357e-08 seconds

LLRBTree: The regression statistics are:
The model of best fit is: y = 5.456361606085358e-07 + 2.3878216299409823e-07*log(x)
The PMCC is: 0.9979169862743112
The coefficient of determination (r^2) is: 0.995838311494804
The RMSE is: 3.194388876716775e-08 seconds

TwoThreeTree: The regression statistics are:
The model of best fit is: y = 4.6815886358961475e-07 + 2.2160944317000748e-07*log(x)
The PMCC is: 0.9930910448978393
The coefficient of determination (r^2) is: 0.9862298234562822
The RMSE is: 5.418941761803715e-08 seconds

Figure 1: Comparison of insertion time under uniformly distributed keys. The 2-3 tree and LLRB tree both show lower insertion cost than the AVL tree, while all three implementations exhibit approximately logarithmic growth with respect to tree size.

2.1 2-3 Tree

Under uniform and normal inputs, the 2-3 tree gave the lowest average insertion times among the three implementations. In the comparison plots, the fitted logarithmic slopes for insertion are approximately 2.22×10^{-7} and 2.21×10^{-7} under uniform and normal data, compared with about 2.38×10^{-7} for the LLRB tree and about 4.0×10^{-7} for the AVL tree. This is consistent with the implementation of 2-3 tree: insertion only triggers local split operations when a node overflows, and there is no need to maintain height data on every recursion return.

The main weakness of the 2-3 tree shown on the experiments is search cost. For successful and unsuccessful searches, the fitted slopes are approximately 2.26×10^{-7} to 2.33×10^{-7} across all four distributions, which is around three times larger than the corresponding AVL and LLRB slopes (roughly 6.3×10^{-8} to 8.4×10^{-8}). The search plots also show a step-like structure, especially for sorted and reverse-sorted data. Long horizontal segments correspond to periods where the tree height remains unchanged, while jumps occur then a new layer is created. The oscillating residuals suggest that even at the same height, performance depends on how full the current level is, i.e. on the mix of 2-nodes and 3-nodes.

Ordered inputs reveal a second weakness, which is poor insertion predictability. Although the mean fitted slope remains competitive, the scatter becomes much larger and the logarithmic fit deteriorates sharply, with R^2 falling to about 0.50 for sorted input and 0.43 for reverse-sorted input. The spikes in the insertion graphs indicate that occasional cascades of split operations through several layers of the tree. Therefore, the 2-3 tree performs well when average insertion throughput matters, but its search overhead and irregular insertion spikes make its runtime less predictable then other two implementations.

With sorted input, every insertion goes to the rightmost leaf. If that leaf is a 2-node, it absorbs the input, turning into a 3-node; if it is a 3-node, the leaf splits, turning back into a 2-node and passing its middle value upward. In the same way, if the parent node is itself a 2-node, it absorbs the middle value and turns into a 3-node, but if it is already a 3-node, it turns into a 2-node and passes on its own middle value. One can imagine, this causes a chain of splits that passes up the right-hand side of the tree, terminating at the first 2-node. More specifically, each insertion causes every 3-node in a vertical chain above (and including) the rightmost leaf to turn into a 2-node. The 2-node at the top of this chain becomes a 3-node. And since each connected 3-node causes a split operation, the insertion causes splits equal in number to number of connected 3-nodes. This works, in a way, like a binary counter. Treating 2-nodes as binary 0s, and 3-nodes as 1s, the number of splits caused by the Nth insertion is simply one less than the number of carries caused by adding one to N when written out in binary. For example, the 7th insertion uses 2 splits, because 7 in binary is 111 and adding 1 to 111 causes 3 carries, and 3 minus 1 is 2.

This succinctly explains the observed 'tree-like' structure. The most expensive inserts for their size are those one less than a power of 2: numbers of the form 1, 11, 111 in binary. Splits only happen on odd insertions. Even numbers have no trailing ones in binary, and do not cause any splits. Half of all binary numbers end in 1. A quarter in 11, an eighth in 111, and so on. So, for every extra split, the spacing between inserts causing that number of splits doubles, explaining the apparent appearance of a binary tree pattern in our plot.

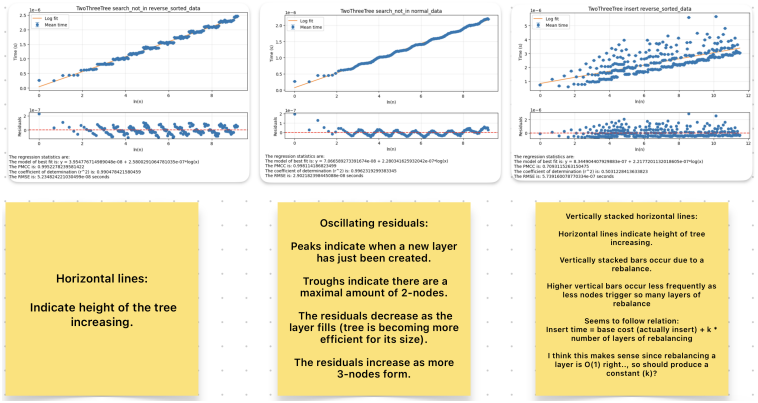


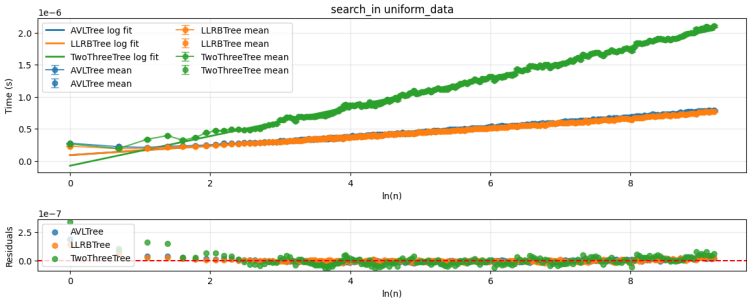
Figure 2: Representative 2-3 tree plots illustrating residual patterns. The step-like timing structure reflects discrete increases in tree height, while oscillating residuals indicate changes in node composition as layers fill and split over time.

2.2 LLRB Tree

The LLRB tree is the most balanced overall implementation. Under uniform and normal inputs, its insertion slopes are about 2.38×10^{-7}, slightly above 2-3 tree and significantly below the AVL tree. Its search performance is also strong, for successful searches the fitted slopes are between 7.14×10^{-8} and 8.06×10^{-8}, and for unsuccessful searches between 6.30×10^{-8} and 8.89×10^{-8}.

This performance is consistent with the structure of LLRB tree. The LLRB tree implements 2-3 tree behaviour using a BST representation, but only using local rotations and color flips for balancing. It avoids the height maintenance performed by the AVL tree, while keeping more efficient than the 2-3 tree.

The limitation of the LLRB tree appears under reverse-ordered insertion. Under the reverse-sorted insertion graph, the fitted slope raises to about 3.03×10^{-7} and R^2 drops to about 0.75. This shows that although the trend remains logmarithmic, the sequence of rotations and color flips becomes more irregular when the inserted elements come with biased order. Overall, the LLRB tree shows the best performance in fast search and cheap insertion, but is less robust than AVL tree under the sorted inputs.



AVLTree: The regression statistics are:
The model of best fit is: y = 9.058825814253881e-08 + 7.394205670120129e-08*log(x)
The PMCC is: 0.9929900380300053
The coefficient of determination (r^2) is: 0.9860292156268313
The RMSE is: 1.8213883831945483e-08 seconds

LLRBTree: The regression statistics are:
The model of best fit is: y = 9.118430440287825e-08 + 7.143083167178729e-08*log(x)
The PMCC is: 0.9953974545549068
The coefficient of determination (r^2) is: 0.9908160925343877
The RMSE is: 1.4231427449032179e-08 seconds

TwoThreeTree: The regression statistics are:
The model of best fit is: y = -7.412023539297023e-08 + 2.3061686427954363e-07*log(x)
The PMCC is: 0.9960245158610402
The coefficient of determination (r^2) is: 0.9920648361962195
The RMSE is: 4.268197580917476e-08 seconds

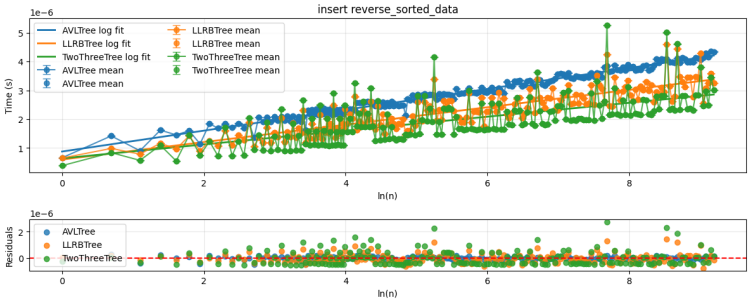
Figure 3: Comparison of successful search time under uniformly distributed keys. AVL and LLRB achieve very similar search performance, whereas the 2-3 tree is consistently slower, despite following the same overall logarithmic trend.

2.3 AVL Tree

The AVL tree has the most stable logarithmic behavior, but it comes with the highest insertion cost. Under uniform and normal inputs, the fitted insertion slopes are about 4.04×10^{-7} and 4.02×10^{-7}, which are significantly higher than LLRB tree and 2-3 trees. Under the sorted and reverse-sorted inputs, AVL tree's insertion slopes fall down to about 3.75×10^{-7} and 3.63×10^{-7}, which means that AVL tree is inserting slower than the other two implementations. This is consistent with its implementation, which re-calculate heights and may perform one or two rotations on the way back up after every insertion.

In contrast, search in the AVL tree is consistently efficient and predictable. Successful search slopes range from about 7.39×10^{-8} to 8.36×10^{-8}, while unsuccessful search slopes range from 6.37×10^{-8} to 7.14×10^{-8}. These slopes are close to the LLRB tree and far below 2-3 tree. The residual plots are also very tight, and the insertion fits under random inputs have very high R^2 values (about 0.998 and 0.997), both indicating that the runtime scales very smoothly to $\log n$.

A significant strength of AVL tree is robustness to adversarial input order. Under reverse-sorted inputs the AVL insertion fit still has $R^2 \approx 0.97$, which is much better than LLRB ($R^2 \approx 0.75$) and the 2-3 tree ($R^2 \approx 0.43$).



AVLTree: The regression statistics are:
The model of best fit is: y = 8.786591017739843e-07 + 3.6280791264798666e-07*log(x)
The PMCC is: 0.9857433156295246
The coefficient of determination (r^2) is: 0.9716898843082885
The RMSE is: 1.2815314368195312e-07 seconds

LLRBTree: The regression statistics are:
The model of best fit is: y = 6.006741766405271e-07 + 3.028059639166439e-07*log(x)
The PMCC is: 0.865281952298821
The coefficient of determination (r^2) is: 0.7487128569740591
The RMSE is: 3.6302576235692255e-07 seconds

TwoThreeTree: The regression statistics are:
The model of best fit is: y = 6.415511982593108e-07 + 2.413005770540014e-07*log(x)
The PMCC is: 0.6577197648720768
The coefficient of determination (r^2) is: 0.43259528910338
The RMSE is: 5.718860051861563e-07 seconds

Figure 4: Comparison of insertion time under reverse-sorted keys. The AVL tree remains the most stable implementation, while the LLRB tree and especially the 2-3 tree show larger variance and more irregular re-balancing costs under highly ordered input.

3 Comparative assessment

(Section 3 should be about one to one-and-a-half pages in length.)

4 Team contributions

(Section 4 should be no more than half a page in length. Express work share as a percentage totalling to 100*: for example, four students who all contributed equally should receive* 25%*.)*

Name	**Portico ID**	**Key contributions**	**Work share (%)**
(name)	(number)	(text)	(xx%)